A Cache in the Weights? Online Learning and Temporal Locality in Small Language Models

Preliminary results

Ruben Tous
Affiliation

Draft, November 5, 2026

Abstract

Attention forgets everything beyond its window, but the text a language model reads at inference time stays on the same topic, characters and vocabulary for far longer. We ask whether a small language model that keeps learning online, one plain SGD step on each chunk after predicting it, uses part of its weights as a cache of its current context. On 25 PG-19 books read by Pythia-14M in independent chunks of 256 tokens, online learning lowers the loss by 0.099 nats/token; for rare tokens last seen in an earlier chunk the gain is +0.301 under 1k tokens back, +0.143 at 1k–10k, and gone beyond 10k. Shuffling the chunks keeps 61% of the overall gain but only 18% of the gain under 1k tokens and 27% at 1k–10k: the long-range gain is local. It is also mostly about the current book: at 1k–10k tokens, the part that needs the original order is +0.117 for tokens last seen in the same book and +0.025 for tokens last seen in an earlier one. A unigram cache of recent tokens is better within 1k tokens but not beyond, and the two combine. The memory lives in the MLPs below the output layer: updating only the MLPs keeps most of the long-range gain, and the MLP of layer 2 alone (0.13M parameters) even gains more than full online learning within 1k tokens, though at a higher learning rate (at the edge of its grid), with a lower overall gain (+0.055 against +0.099) and a larger loss on never-seen tokens. The effect is as large at 31M parameters as at 14M and almost vanishes at 70M. These are preliminary results: one seed, plain SGD with per-run learning rates, short books, three model sizes.

1 Introduction

What a model reads at inference time has strong temporal locality. A book keeps its characters, places and turns of phrase for hundreds of thousands of tokens; a conversation, a code base or a document collection behaves the same way. Computer memory hierarchies have long exploited this property with a small cache that holds the recently used data [5]. A transformer language model has such a cache in its attention window, but the window is short: anything older than the window is forgotten.

Weights updated online could keep a memory of the current context beyond the window. Dynamic evaluation [10, 11, 15] and recent test-time training methods [6, 2, 19] do exactly this and report lower perplexity or better long-context performance. What is less clear is what kind of memory the updated weights hold. If online learning turns part of a model into a cache, the gain should be largest for material seen recently and fall with distance (recency), new material should push old material out (eviction, the useful side of the catastrophic interference that sequential training causes [13]), and the general knowledge in the weights should survive. Alternatively, the gain may come mostly from adapting to the domain or style of the text in general, which needs no memory of the recent past at all.

We study this question with a deliberately simple setup and a small model, where controls are cheap. Pythia-14M [4] reads 25 PG-19 test books [14] in chunks of 256 tokens scored with no context from the previous chunk, so anything that helps beyond 256 tokens has to come through the weights; after scoring each chunk, it takes one SGD step on it.

Our contributions are the following.

- A **shuffled-stream control** that separates the local part of the online gain (which needs the recent material) from the global part (adapting to the text in general), measured per token and per distance bin.

- A **count-cache baseline**: a unigram cache of recent tokens mixed into the frozen model, and its combination with online learning.
- A **distance and same-book analysis**: the long-range gain is mostly about being in the same book, not only about recency.
- The **location of the memory** in the network: updating only the MLPs, or a single layer's MLP, keeps most of the long-range gain; the output layer alone does not.
- Preliminary evidence on **eviction** and on **model size** (14M, 31M, 70M), with their limitations.

2 Related work

Dynamic evaluation. Krause et al. [10] adapt a recurrent language model to the recent history of the sequence it is predicting with gradient steps at test time, exploiting re-occurring patterns; applied to Transformer-XL the gains are smaller than for weaker models, which the authors take to suggest that Transformers already model re-occurring patterns better than earlier architectures [11]. Rannen-Triki et al. [15] revisit dynamic evaluation for larger models and frame it as memory in weights, a form of context-length extension, with the largest gains under distribution shift. We use the same prequential protocol (score, then update) and ask what the updated weights remember and where.

Cache language models. Kuhn and De Mori [12] mix a cache of recently seen words into an n-gram model; Grave et al. [9] store recent hidden states and their next words and mix the resulting distribution into a neural model without retraining. Our unigram cache is the simplest member of this family and serves as the baseline that any "cache in the weights" should beat.

Test-time training for language models. Test-time training adapts a model to each test input with a self-supervised loss [18]. For language models, In-Place TTT [6] treats the final projection of the MLP blocks as fast weights updated with a next-token objective; qTTT [2] updates only the query projections on the long context; TTT-E2E [19] meta-learns a model for test-time training on long context. Our finding that the MLPs below the output hold the memory is consistent with In-Place TTT's choice of MLP weights as the adaptable part, and with the view of feed-forward layers as key-value memories [8].

Fast weights. Fast weights change at inference time to hold short-term memory [17, 1], and linear transformers can be read as fast weight programmers [16]; online SGD on a pretrained model is a crude, untrained form of fast weights.

What this work adds. Prior work reports the overall gain of online adaptation; we split it by distance, into local and global parts, by book, against a count cache, and by the part of the network that is updated.

3 Setup

Models. Pythia-14M [4] (14.07M parameters: 6 layers of width 128, plus untied input and output embeddings of 6.44M parameters each), and for the size comparison Pythia-31M (30.49M) and Pythia-70M (70.43M), all with 6 layers and the same tokenizer; float32 on CPU.

Data. PG-19 test books [14], in the order the Hugging Face dataset gives them. Books with fewer than 20k tokens (three single short stories) are skipped; each remaining book is cut to its first 20k tokens. The first 25 books, concatenated, form the stream (500k tokens); the next 2 books are *check books*, used only to pick learning rates. Pythia was trained on the Pile [7], whose PG-19 component has 28,602 documents [3], exactly the size of PG-19's training split; the test books are therefore very likely not in Pythia's training data, although copies elsewhere in the Pile were not checked.

bin (previous occurrence)	tokens	frozen loss	online, in order	online, shuffled	locality
all tokens	498046	4.3922	+0.099	+0.061	+0.038
same chunk	9881	5.0194	−0.107	−0.138	+0.030
earlier chunk, < 1k	17054	7.3382	+0.301	+0.054	+0.247
earlier chunk, 1k–10k	34076	7.0001	+0.143	+0.038	+0.105
earlier chunk, 10k–100k	44133	7.1181	+0.010	+0.023	−0.014
earlier chunk, ≥ 100k	13178	7.8193	−0.035	−0.018	−0.017
never seen	20959	8.1856	−0.062	−0.055	−0.007

Table 1: Gain over the frozen model (nats/token) by distance to the token's previous occurrence, Pythia-14M, all parameters updated online (lr 10^{-4}). Shuffled: the same chunks in a random order. Locality = in order − shuffled. The 1000 most frequent token ids are excluded from the bins, not from "all tokens".

Protocol. The stream is cut into non-overlapping chunks of $L = 256$ tokens. Each chunk is scored on its own, with no attention to earlier chunks; every token but the chunk's first is predicted from the tokens before it in the chunk. Online, after scoring a chunk the model takes one plain SGD step (no momentum, no weight decay) on the mean loss of that chunk, which is a prequential protocol: every token is predicted before it is trained on. The frozen model reads the same stream without updates. Some runs update only part of the network: the output embedding, everything but the output embedding, the MLPs or the attention of all layers, or the MLP of one layer.

Learning rates. Each run's learning rate is the one with the lowest mean loss on the 2 check books, from a small grid (three values); from the last test on, a pick at the edge of its grid extends the grid once by one step. For Pythia-14M with all parameters the pick is 10^{-4} (check-book loss 4.4775, against 4.5474 frozen).

Metrics. The *gain* of a run is the frozen loss minus the run's loss over the same tokens, in nats/token; each model is compared with its own frozen loss. For each scored token we find the previous occurrence of the same token id anywhere earlier in the stream and bin the token by where it is: in the *same chunk* (attention can see it), in an *earlier chunk* under 1k, 1k–10k, 10k–100k or at least 100k tokens back, or *never seen*. The 1000 most frequent token ids in the stream are left out of the bins, since they are always recent (139,281 of the 498,046 scored tokens remain). Earlier-chunk bins are further split by whether the previous occurrence is in the token's own book or an earlier one. The *locality* is the gain in the original order minus the gain on the same chunks in a shuffled order (one random permutation, seed 0), per token: the part of the gain that needs the recent material. For eviction, the *memory of A* is defined in Section 4.5.

4 Results

4.1 The online gain reaches beyond the window and fades by 10k tokens

Online learning lowers Pythia-14M's loss from 4.3922 to 4.2929 nats/token (gain +0.099, Table 1, Figure 1a); for rare tokens last seen in an earlier chunk, the gain falls from +0.301 under 1k tokens back to −0.035 beyond 100k. Never-seen tokens lose (−0.062): online learning moves probability towards recent material at the expense of the rest. Tokens repeated inside their own chunk also lose (−0.107): attention already copies them, and the updates work against that. After each switch to a new book the gain starts near zero and grows with the position in the book, from +0.009 in the first 1k tokens to +0.080 at 3k–4k and +0.145 at 19k–20k, with no plateau within the 20k tokens of a book (Figure 1b).

4.2 The long-range gain is local

Shuffling the chunks across all books keeps the training data and removes the order. The shuffled learner still gains +0.061 overall, 61% of the in-order gain: most of the overall gain is adaptation to old books in general. But the gain beyond the window needs the order: under 1k tokens back only 18% survives shuffling (+0.054 of +0.301, locality +0.247), and at 1k–10k only 27% (+0.038 of +0.143, locality +0.105). Beyond 10k the shuffled learner is slightly better (locality −0.014 and −0.017): the in-order learner uses nothing that far back.

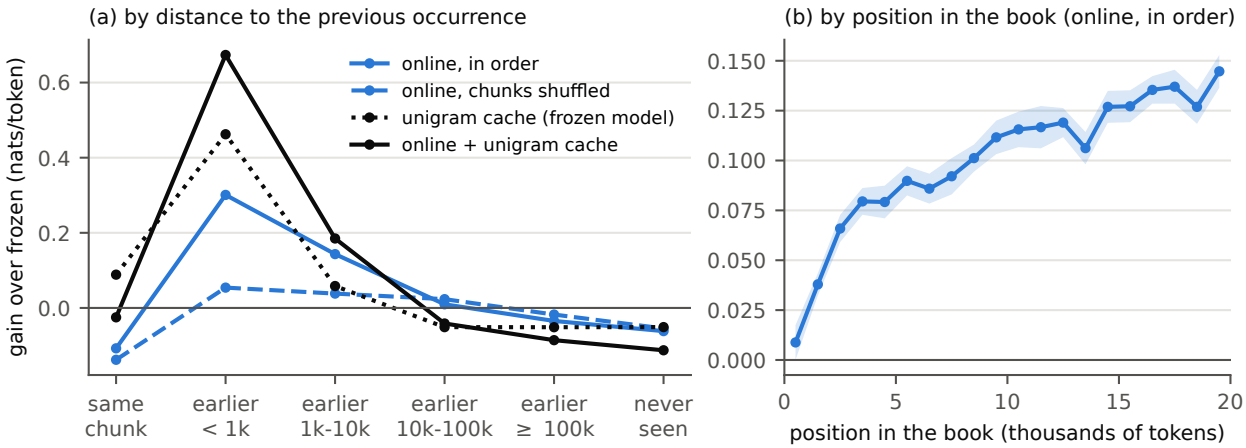


Figure 1: (a) Gain over frozen by distance bin for online learning in order and with shuffled chunks, the unigram cache on the frozen model, and online learning combined with the cache. (b) Gain of online learning (in order) by position in the book, mean over the 24 books that follow a book switch, ±1 standard error over books.

bin	previous occurrence	tokens	in order	shuffled	locality	unigram cache
earlier chunk, < 1k	same book	16988	+0.302	+0.054	+0.248	+0.463
earlier chunk, 1k–10k	same book	29680	+0.155	+0.038	+0.117	+0.069
	earlier book	4396	+0.065	+0.040	+0.025	−0.016
earlier chunk, 10k–100k	same book	4814	+0.065	+0.027	+0.038	−0.050
	earlier book	39319	+0.003	+0.023	−0.020	−0.051

Table 2: Gain over frozen (nats/token) split by whether the previous occurrence is in the same book or an earlier one, Pythia-14M. The earlier-book part of the < 1k bin (66 tokens) and the ≥ 100k bin (all earlier book) are omitted.

4.3 It is mostly about being in the same book

Table 2 splits each earlier-chunk bin by whether the previous occurrence is in the same book (books are 20k tokens long, so at most 20k tokens back) or in an earlier book. At 1k–10k tokens back, the locality is +0.117 for same-book tokens and +0.025 for earlier-book tokens. Same-book tokens 10k–20k back still have a locality of +0.038, more than earlier-book tokens that are closer (1k–10k), while earlier-book tokens at 10k–100k have none (−0.020). The fall of the gain with distance in Table 1 is therefore largely the fall in the share of same-book tokens, although distance still matters within a book (locality +0.248, +0.117 and +0.038). The unigram cache depends more on distance than on the book: at 1k–10k it gains +0.069 on same-book tokens and −0.016 on earlier-book ones, but already at 10k–20k within the same book it loses (−0.050), where online learning still gains +0.065.

4.4 More than a count cache, and complementary to it

Table 3 compares online learning with a unigram cache on the frozen model, $p = (1-\lambda)\, p_{\text{model}} + \lambda\, p_{\text{cache}}$, where p_{cache} holds the normalized, exponentially decaying counts of the token ids seen so far, updated after each chunk like the online learner ($\lambda = 0.05$, half-life 1k tokens, picked on the check books). The cache is the better short-range copier (+0.462 under 1k tokens, +0.089 within the chunk); from 1k tokens on online learning is better (+0.143 against +0.058 at 1k–10k), and overall it gains six times more. The two combine: mixing the same cache into the online learner's predictions gives +0.673 under 1k and +0.185 at 1k–10k. Beyond 10k tokens and on never-seen tokens the combination loses more, so overall it is only slightly better than online learning alone (+0.102 against +0.099); the mixing weight was picked for the frozen model.

Updating everything but the output embedding reproduces the full learner in every bin (+0.094 overall). Updating only the output embedding is a short-range cache at best: at 3×10^{-3} (at the edge of its grid) it gains

bin	online, all parameters	unigram cache	online + cache	output only (lr 10^{-3})	output only (lr 3×10^{-3})	all but output
all tokens	+0.099	+0.017	+0.102	+0.045	+0.072	+0.094
same chunk	−0.107	+0.089	−0.025	+0.011	+0.092	−0.105
earlier chunk, < 1k	+0.301	+0.462	+0.673	+0.061	+0.251	+0.290
earlier chunk, 1k–10k	+0.143	+0.058	+0.185	−0.001	+0.066	+0.139
earlier chunk, 10k–100k	+0.010	−0.051	−0.041	−0.039	−0.054	+0.009
earlier chunk, ≥ 100k	−0.035	−0.051	−0.086	−0.075	−0.146	−0.034
never seen	−0.062	−0.051	−0.113	−0.064	−0.143	−0.058

Table 3: Gain over frozen (nats/token) of online learning, a unigram cache on the frozen model, their combination, and online learning restricted to the output embedding or to everything else (Pythia-14M; lr 10^{-4} unless stated).

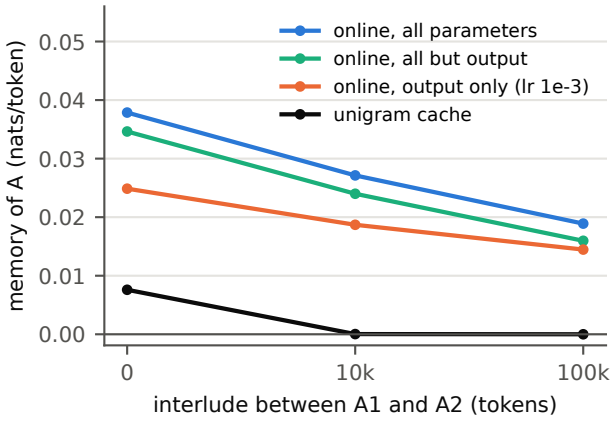


Figure 2: Memory of book A (loss on A2 after B1 minus loss on A2 after A1) against the number of tokens of other books between A1 and A2, mean of 3 books, Pythia-14M.

+0.251 under 1k tokens but only +0.066 at 1k–10k, and at 10^{-3} almost nothing that depends on distance. The long-range gain lives below the output layer.

4.5 Eviction: a book's memory decays slowly

To measure eviction we split three books A (a memoir, a novel and a travel book) into halves A1 and A2 of 10k tokens. Online learning runs on A1, then an interlude of $I \in \{0, 10\text{k}, 100\text{k}\}$ tokens of other books, then A2; a second run replaces A1 by the first 10k tokens of a different book B1. Both runs have adapted to books in general by the time they reach A2; only the first has seen A. The *memory of A* is the loss on A2 after B1 minus the loss on A2 after A1.

With all parameters updated, the memory of A is +0.038 nats/token at $I = 0$, +0.027 after 10k tokens and +0.019 after 100k (mean of the 3 books; all three are positive at every I; Figure 2). About 44% of the A1 run's gain on A2 at $I = 0$ (+0.086) is specific to the book. The comparison is matched (the same interlude, and A1 and B1 of the same length), so the memory of A halves over 100k tokens of other books. How the two runs get there is open to two readings. The A1 run's gain on A2 is flat (+0.086, +0.082, +0.085) while the B1 run's grows (+0.048, +0.055, +0.067). This does not show that the A1 run keeps its memory of A: it is equally consistent with the A1 run losing A-specific memory while gaining general adaptation from the interlude at the same rate as the B1 run, and this test cannot separate the two. Updating everything but the output embedding gives a similar memory (+0.035, +0.024, +0.016); output only (lr 10^{-3}) starts lower (+0.025) and decays more slowly (+0.014 at 100k); the unigram cache keeps +0.008 at $I = 0$ and nothing after 10k tokens. Book-level memory thus lasts much longer than the token-level gain of Table 1: what lasts may be a book's style and vocabulary rather than specific tokens.

bin	all	MLP	MLP	MLP of layer					
	params	all 6	last 2	1	2	3	4	5	6
learning rate	10^{-4}	10^{-4}	10^{-3}	10^{-3}	3×10^{-3}	3×10^{-4}	3×10^{-3}	3×10^{-3}	10^{-3}
all tokens	+0.099	+0.085	+0.070	+0.052	+0.055	+0.055	+0.067	+0.072	+0.047
same chunk	−0.107	−0.086	−0.076	−0.079	−0.137	−0.088	−0.093	−0.076	−0.053
< 1k	+0.301	+0.216	+0.225	+0.229	+0.395	+0.202	+0.357	+0.285	+0.108
1k–10k, same book	+0.155	+0.130	+0.104	+0.115	+0.133	+0.093	+0.135	+0.120	+0.052
10k–100k, same book	+0.065	+0.063	+0.029	+0.049	+0.002	+0.026	+0.015	+0.028	+0.009
never seen	−0.062	−0.055	−0.095	−0.054	−0.129	−0.058	−0.116	−0.090	−0.084

Table 4: Gain over frozen (nats/token) when only some MLPs of Pythia-14M are updated online, in order. Distance bins are earlier-chunk bins. Learning rates were picked on the check books; those at 3×10^{-3} are at the edge of the extended grid.

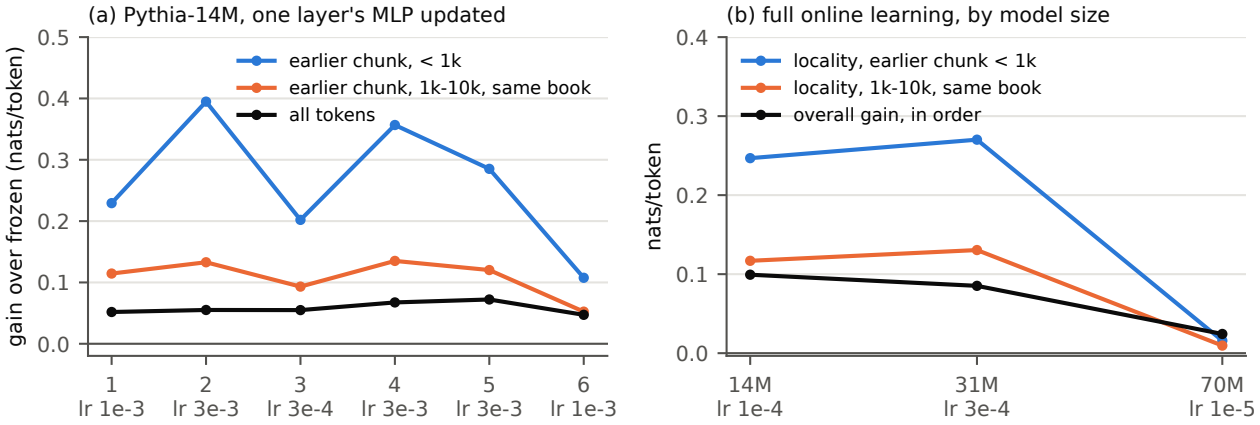


Figure 3: (a) Gain of online learning restricted to the MLP of one layer of Pythia-14M, with the learning rate picked per layer. (b) Locality (gain in order minus gain shuffled) and overall gain of full online learning for Pythia-14M, 31M and 70M, each at its own learning rate.

4.6 The memory lives in the MLPs

Updating only the MLPs of all six layers (0.79M parameters, lr 10^{-4}) keeps most of the long-range gain (+0.216 under 1k tokens and +0.123 at 1k–10k, 72% and 86% of the full learner); updating only attention (0.40M) keeps less (+0.142, +0.092). Table 4 goes further: the MLP of a single layer (131,712 parameters), with the learning rate picked per layer. Every layer's MLP but the last holds a large part of the memory, and the middle ones the most: the MLP of layer 2 or 4 alone gains +0.395 and +0.357 under 1k tokens, more than the full learner (+0.301), and +0.133 and +0.135 at 1k–10k in the same book (full learner +0.155; Figure 3a). The last layer's MLP is the weakest (+0.108, +0.052). The single-layer runs use higher learning rates (up to 3×10^{-3}, at the edge of the grid) and pay for their larger short-range gains with larger losses on tokens not seen recently (layer 2: −0.129 on never-seen tokens). With tuned rates, the MLPs of the last two layers together keep +0.225 under 1k tokens, about what all six keep at 10^{-4}; an earlier run at 10^{-4} had lost almost all of it, which was under-tuning.

4.7 Model size: flat from 14M to 31M, a drop at 70M

Table 5 repeats the in-order and shuffled runs for Pythia-31M and 70M, each at its own learning rate. The locality does not fall steadily with size (Figure 3b). 31M matches or slightly exceeds 14M: +0.270 against +0.247 under 1k tokens, +0.131 against +0.117 at 1k–10k in the same book. At 70M it almost vanishes (+0.016 and +0.010), and the overall gain drops to +0.024. The drop is not a matter of a missing learning rate in the grid: 70M's best rate, 10^{-5}, lowers its check-book loss by only 0.011 nats/token (31M: 0.057 at 3×10^{-4}; 14M: 0.070 at 10^{-4}), and an earlier run at 3×10^{-5} had a larger locality (+0.047) but a worse check-book loss. With one seed and three sizes, the pattern looks more like something specific to how 70M responds to plain SGD than a

	Pythia-14M		Pythia-31M		Pythia-70M	
learning rate	10^{-4}		3×10^{-4}		10^{-5}	
frozen loss (all tokens)	4.3922		4.0404		3.8185	
bin	in order	locality	in order	locality	in order	locality
all tokens	+0.099	+0.038	+0.085	+0.039	+0.024	+0.004
same chunk	−0.107	+0.030	−0.102	−0.012	−0.039	−0.018
earlier chunk, < 1k	+0.301	+0.247	+0.373	+0.270	+0.048	+0.016
1k–10k, same book	+0.155	+0.117	+0.171	+0.131	+0.034	+0.010
1k–10k, earlier book	+0.065	+0.025	+0.050	+0.037	+0.026	+0.008
10k–100k, same book	+0.065	+0.038	+0.059	+0.038	+0.019	+0.005
never seen	−0.062	−0.007	−0.041	−0.020	−0.037	−0.014

Table 5: Gain over each model's own frozen loss (nats/token) and locality, full online learning.

smooth trend.

5 Discussion and limitations

The results support a qualified version of the cache picture. Online learning gives a small language model a memory beyond its attention window, and the long-range part of it is local. But it behaves less like a cache of recent tokens than like a memory of the current book: its gain follows book membership more than distance, the memory of a book is still half its size after 100k tokens of other books, and a count cache does the short-range copying better. The general knowledge in the weights is only partly kept: tokens not seen recently are predicted slightly worse (−0.062 for never-seen tokens), the cost of moving probability towards the current context. That the memory sits in the MLPs, and that a single middle-layer MLP can carry much of it (at a higher learning rate and a larger cost on other tokens), fits the view of feed-forward layers as key-value memories [8] and the choice of MLP weights as fast weights in recent test-time training [6].

These are preliminary results with clear limitations.

- **One seed, few books.** One seed and one chunk order per run; three books in the eviction test. Differences of a few thousandths of a nat are within noise.
- **Plain SGD, per-run learning rates** picked from three or four values on two books, several at the edge of their grid. Comparisons across layers and sizes mix in the learning rate.
- **Short books** (20k tokens): same-book memory beyond 20k tokens and the plateau of the per-book gain are not observed.
- **Three model sizes, non-monotonic.** The collapse at 70M may come from the optimizer rather than from size.
- **Capacity not tested.** The eviction test cannot separate forgetting a book from adapting to books in general.
- **Contamination** is very likely absent from the Pile's PG-19 component, but the rest of the Pile was not checked.

Next steps follow directly. Interleaving K books chunk by chunk, with K growing, would measure the capacity of the weights as a cache. An update normalized across sizes (per layer or per parameter, or a fixed relative weight change) would make the size comparison fairer, and more sizes would show whether the drop at 70M is a trend. PolyPythias [20], which provides several pre-training seeds per Pythia size, would give error bars across models.

6 Conclusion

A small language model that takes one SGD step on each chunk it reads gains a memory beyond its attention window. In Pythia-14M on PG-19 books this memory is local, tied mostly to the current book rather than to

the recency of individual tokens, stored in the MLPs below the output layer, complementary to a count cache, and slow to fade when other books follow. It is as strong at 31M parameters and nearly absent at 70M under plain SGD. Whether it behaves like a cache with a measurable capacity is the next question.

References

[1] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems 29 (NIPS)*, 2016.

[2] Rachit Bansal, Aston Zhang, Rishabh Tiwari, Lovish Madaan, Sai Surya Duvvuri, Devvrit Khatri, David Brandfonbrener, David Alvarez-Melis, Prajjwal Bhargava, Mihir Sanjay Kale, and Samy Jelassi. Let's (not) just put things in context: Test-time training for long-context LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2512.13898.

[3] Stella Biderman, Kieran Bicheno, and Leo Gao. Datasheet for the Pile. arXiv:2201.07311, 2022.

[4] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 2397–2430, 2023.

[5] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[6] Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Di He, Wenhao Huang, and Tianle Cai. In-place test-time training. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2604.06169.

[7] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling. arXiv:2101.00027, 2020.

[8] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5484–5495, 2021.

[9] Edouard Grave, Armand Joulin, and Nicolas Usunier. Improving neural language models with a continuous cache. In *International Conference on Learning Representations (ICLR)*, 2017.

[10] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of neural sequence models. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 2766–2775, 2018.

[11] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of transformer language models. arXiv:1904.08378, 2019.

[12] Roland Kuhn and Renato De Mori. A cache-based natural language model for speech recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(6):570–583, 1990.

[13] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[14] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

[15] Amal Rannen-Triki, Jorg Bornschein, Razvan Pascanu, Marcus Hutter, András György, Alexandre Galashov, Yee Whye Teh, and Michalis K. Titsias. Revisiting dynamic evaluation: Online adaptation for large language models. arXiv:2403.01518, 2024.

[16] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 9355–9366, 2021.

[17] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.

[18] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 9229–9248, 2020.

[19] Arnuv Tandon, Karan Dalal, Xinhao Li, Daniel Koceja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, Carlos Guestrin, Jed McCaleb, Yejin Choi, and Yu Sun. End-to-end test-time training for long context. arXiv:2512.23675, 2025.

[20] Oskar van der Wal, Pietro Lesci, Max Müller-Eberstein, Naomi Saphra, Hailey Schoelkopf, Willem Zuidema, and Stella Biderman. PolyPythias: Stability and outliers across fifty language model pre-training runs. In *International Conference on Learning Representations (ICLR)*, 2025.